

# Sztuczna inteligencja. Przeszukiwanie

Paweł Rychlikowski

Instytut Informatyki UWr

5 marca 2026

# Słowa klucze dla naszego wykładu (przypomnienie)

- Przeszukiwanie (problem solving)
- Uczenie się (learning)
- Wnioskowanie (inference, theorem proving)
- Modelowanie wiedzy o świecie

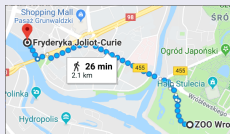
# Słowa klucze dla naszego wykładu (przypomnienie)

- **Przeszukiwanie (problem solving)**
- Uczenie się (learning)
- Wnioskowanie (inference, theorem proving)
- Modelowanie wiedzy o świecie

Jakieś elementy przeszukiwania występują również w uczeniu, używaniu modeli i wnioskowaniu

# Problem solving by searching. Intuicje

## Przykład 1. Wyznaczanie trasy



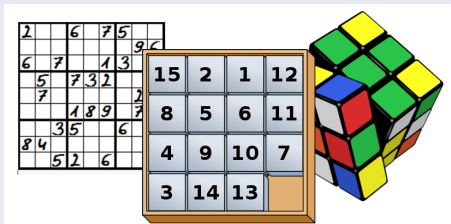
## Przykład 2. Wyznaczanie sekwencji działań

Kohler (1945): monkey and banana problem.



Kohler observed that chimpanzees appeared to have an insight into the problem before solving it

## Przykład 3. Rozwiązywanie łamigłówek



## Uwaga

**Problemy zabawkowe** (toy problems) są częstym narzędziem w AI.

# Czym jest problem?

## Definicja

**Problem** ma następujące pięć komponentów

1. Stan początkowy (i zbiór stanów, ale być może dany implícite)
2. Zbiór akcji (co **agent** może robić)
3. Model przejścia ( $\text{stan} + \text{akcja} = \text{nowy\_stan}$ )
4. Test określający, czy stan jest końcowy (i znaleźliśmy rozwiązanie)
5. Sposób obliczania kosztu ścieżki (najczęściej podawany jako koszt akcji w stanie)

**Agent** – program, który odbiera wrażenia o świecie, podejmuje decyzje, wykonuje akcje, maksymalizując jakąś funkcję celu.

# Piętnastka



- **Stan początkowy:** jakieś ułożenie, na przykład powyższe
- **Stan końcowy:** liczby po kolei, pusty kwadrat w lewym górnym rogu
- **Koszt:** jednostkowy
- **Model:** dowolna zamiana pustego kwadracika z sąsiadem

# Zadanie szachowe z pierwszej listy

- **Stan początkowy**: jakieś ułożenie wieży i dwóch króli, informacja o tym, kto się rusza
- **Stan końcowy**: mat
- **Koszt**: jednostkowy
- **Model**: ruch szachowy + zmiana gracza aktywnego



# Modyfikacje zadania znajdowania marszruty

## Wariant 1

W co  $K$ -tej miejscowości na trasie znajduje się dobra knajpa.

## Wariant 2

Powinniśmy *zaliczyć* co najmniej  $K$  dodatkowych atrakcji turystycznych.

Zastanówmy się, co jest stanem w powyższych zadaniach?

# Modyfikacje zadania znajdowania marszruty

## Wariant 1

W co  $K$ -tej miejscowości na trasie znajduje się dobra knajpa.

Stan: (miejscowość, licznik-modulo- $K$ )

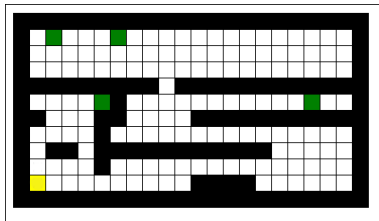
## Wariant 2

Powinniśmy zaliczyć co najmniej  $K$  dodatkowych atrakcji turystycznych.

Stan (1): (miejscowość, liczba-atrakcji-do-zaliczenia) **źle!**

Stan (2): (miejscowość, zbiór-zaliczonych-atrakcji) **lepiej!**

Stan (3): (jaki?)



- Będziemy teraz rozważać różne labirynty, na kwadratowej siatce.
- Labirynt jako problem wyszukiwania:
  - **stan** – współrzędne pola na którym można stanąć (nie ściany)
  - **start** – ustalona pozycja w labiryncie (żółta)
  - **cel** – ustalone pozycje w labiryncie (zielona)
  - **model** – 4-sąsiedztwo (modulo ściany), akcje to N, W, E, S.
  - **koszt** – jednostkowy

## Labirynt 2. Możliwe modyfikacje

Można wzbogacić przestrzeń stanów w labiryncie

- Dodać drzwi i klucze (czym stanie się **stan**)?
- Dodać poruszających się (deterministycznie) wrogów (**stan**?)
- Dodać skrzynie z bronią, apteczki i punkty życia (**stan**?)

## Uwaga

Akcje oraz model przejścia są ze sobą powiązane. Jest kilka wariantów:

1. Zbiór akcji wspólny, funkcja przejścia pilnuje, żeby **niemożliwe** akcje nie zmieniały stanu  
Przykład: ludzik idzie w labiryncie na ścianę
2. Akcje to funkcja, która dla stanu zwraca zbiór czynności, które agent może zrobić w stanie
3. Akcja i model przejścia to jedna funkcja, która dla stanu zwraca **listę** par postaci:  
(akcja, nowy\_stan)

# Poziomy abstrakcji

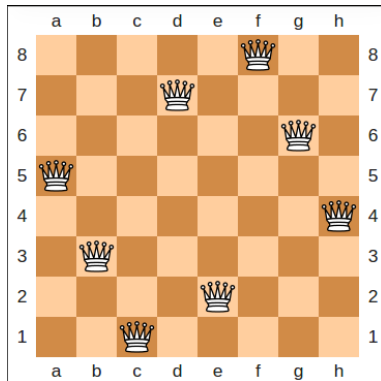
Zadania z rzeczywistego świata można modelować na wiele różnych sposobów. Dla **podróżowania MPK** mamy następujące akcje/sekwencje akcji

- Przejdź z przystanku A do przystanku B
- Wsiądź do tramwaju na przystanku A, skasuj/zakup bilet, zajmij wygodne miejsce, obserwuj tablicę, podejdź do drzwi, gdy...
- Wykonuj naprzemienne ruchy lewą i prawą nogą, aż ...
- Napnij głowę większą mięśnia dwugłowego lewego uda, ...

## Uwaga

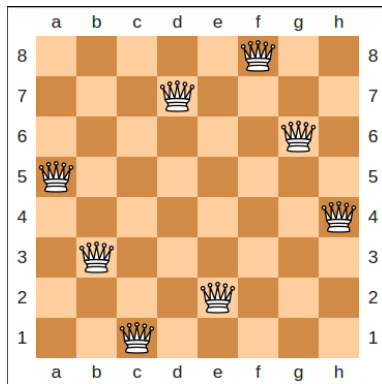
Akcje muszą być zrozumiałe dla agenta, im wyższy poziom, tym łatwiejsze zadanie przeszukiwania.

# Problem ośmiu hetmanów



- W przeciwieństwie do poprzedniego przykładu istnieje tu wiele sformułowań.
- Jakie są dwie najważniejsze opcje?

# Problem ośmiu hetmanów



Dwa sposoby opisywania zadania jako problemu przeszukiwania:

1. **Stan kompletny** (rozważamy sytuacje z 8 hetmanami na planszy, ruch to przestawienie hetmana)
2. **Stan niepełny** – zaczynamy od pustej planszy, ruchem jest postawienie hetmana.



## Uwaga

Rozmieszczamy hetmany wg określonego schematu, rozpoczynając od pustej planszy. Liczymy liczbę stanów.

- $64 \times 63 \times \dots \times 57 = 178462987637760$  – rozmieszczamy po kolei hetmany, na jednym polu jest 1 hetman.
- $8^8 = 16777216$  – ustalona kolejność, w każdej kolumnie 1 hetman
- $8! = 40320$  – ustalona kolejność, w każdej kolumnie 1 hetman, nie szachują się w wierszach

# Nieskończona przestrzeń: dowodzenie twierdzeń

- **Stany** to zbiory **lematów** (uprzednio dowiedzionych faktów)
- **Stan początkowy**: zbiór aksjomatów
- **Model**: Zbiór reguł wyprowadzania nowych twierdzeń na podstawie twierdzeń już dowiedzionych
- **Stan końcowy**:: zbiór twierdzeń zawierających twierdzenie  $\Psi$ , które właśnie dowodzimy

Przykładowe zadania z rzeczywistego świata, będące zadaniami przeszukiwania:

- Różne zagadnienia logistyczne
  - Projektowanie układów scalonych
  - Nawigacja robotów
  - Organizacja procesu produkcji (minimalizująca koszty, albo ilość szkodliwych odpadów)
  - Tworzenie projektów (na przykład leków) o określonych właściwościach
- 
- w przestrzeni parametrów sieci neuronowej znajdź te właściwe,
  - w modelu generatywnym (ChatGPT, DALLÉ-2, ...) znajdź odpowiedź, spełniającą specyfikacje

# Oczekiwania wobec rozwiązania

(wracamy do ogólniejszych zagadnień)

- **Zupełność**: czy program znajdzie drogę do rozwiązania, jeżeli takowa istnieje?

Czy to jest konieczny warunek użyteczności algorytmu?

- **Optymalność**: czy będzie ona najkrótsza
- **Złożoność czasowa**: jak długo będzie trwało szukanie
- **Złożoność pamięciowa**: ile zużyjemy pamięci

## Pytanie

Dlaczego tak ważna jest złożoność pamięciowa?

- Agent **bez pamięci** z konieczności operuje drzewem przeszukiwań (bo nie potrafi stwierdzić, że w jakimś stanie już był)
- Najczęściej lepiej modelować świat za pomocą **grafu**

**BFS** = Breadth First Search

## Opis

- Mamy 3 grupy stanów: **do-zbadania**, **zbadane** i pozostałe.
- Na początku mamy 1 stan **do-zbadania**: stan startowy
- Stany do zbadania przechowujemy w kolejce **FIFO** (first-in first out)
- Badanie stanu:
  - Sprawdzenie, czy jest stanem docelowym (jak tak, to **koniec!**)
  - Ustalenie, jakie akcje możemy zrobić w tym stanie, znalezienie nowych stanów **do-zbadania**

## Skrócony opis

Pobieraj stan z **kolejki**, przetwarzaj, jak kolejka się skończy (nie **do-zbadania**) to zakończ działanie, (możesz też zakończyć, jak znajdziesz stan docelowy).

**DFS** = Depth First Search

## Opis

- Stany przetwarzamy w innej kolejności: dzieci aktualnie rozwijanego mają priorytet
- Czyli zamiast FIFO używamy LIFO (List in First out), czyli po prostu stosu.
- Oprócz tego algorytm się nie zmienia.

**DLS** = Depth Limited Search

## Opis

- Określamy maksymalną głębokość poszukiwania.
- Przeszukujemy w głąb, ale nie rozwijamy węzłów na głębokości większej niż  $L$ .
- Wygodnie implementuje się rekurencyjnie (proste ćwiczenie)



- W algorytmach na grafach używa się takich parametrów jak  $|V|$  oraz  $|E|$  (liczba stanów, liczba krawędzi)
- Dobra złożoność to może być  $O(|V| + |E|)$
- W sztucznej inteligencji, gdzie często nie znamy grafu (lub jest on zbyt duży, żeby traktować go jako daną do zadania), używamy innych parametrów

# Analiza czasowo pamięciowa (2)

## Parametry zadania wyszukiwania

- $b$  – maksymalne rozgałęzienie (branching factor)
- $d$  – głębokość najpłytszego węzła docelowego
- $m$  – maksymalna długość ścieżki w przestrzeni poszukiwań

## Uwaga

Mówiąc o czasie (pamięci) często używamy jako jednostki liczby węzłów (przetworzonych/pamiętanych).

# Czas i pamięć dla BFS i DFS

## BFS

$$\text{Czas} = (O(b + b^2 + b^3 + \dots + b^d) = O(b^d))$$

$$\text{Pamięć} = \text{Czas}$$

**Uwaga:** Może być też  $O(b^{d+1})$  jak testujemy warunek sukcesu dopiero podczas rozwijania.

## DFS

$$\text{Czas} = O(b^m) - \text{niedobrze}$$

$$\text{Pamięć} = O(bm) - \text{dobrze}$$

## Uwaga

W tych rozważaniach zakładamy, że przestrzeń jest tak wielka, że nie spamiętujemy odwiedzonych stanów (względnie wiemy, że stany się nie powtarzają)

Oczywiście w skończonych grafach nasze rozważania są nazbyt pesymistyczne!

## Uwaga

**Iteracyjne pogłębianie** to po prostu wywoływanie DLS na coraz to większej głębokości (bez zapamiętywania żadnych pośrednich wyników)

Może wydawać się to stratą czasu, ale:

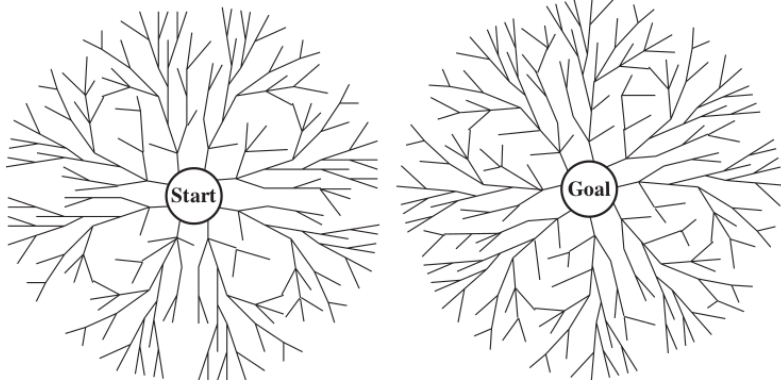
- działamy w pamięci  $O(bd)$ ,
- na czas wpływa ostatnia warstwa, czyli  $O(b^d)$

# Przeszukiwanie dwukierunkowe

## Pomysł

Prowadźmy poszukiwania jednocześnie od przodu i od tyłu

Rysunek:



# Przeszukiwanie dwukierunkowe. Problemy i korzyści

## Problemy

Nie zawsze jest możliwe do zastosowania:

1. Musimy znać stan końcowy (vide hetmany czy obrazki logiczne)
2. Najlepiej jak jest jeden (albo niewiele i umiemy je wszystkie wymienić)
3. Musimy umieć odwrócić funkcję następnika
4. Musimy pamiętać odwiedzone stany (przynajmniej z jednej strony)

**BFS + IDS** (lub **BFS + BFS**) zamiast **IDS+IDS**

## Korzyści

Podstawowa korzyść to czas działania. Dlaczego?

Odpowiedź: **Zamiast jednego przeszukania na głębokości  $d$  mamy dwa przeszukania na głębokości  $d/2$ .**